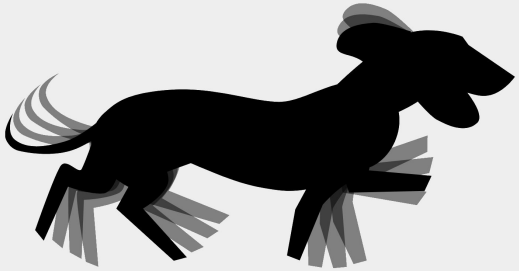


ARGO

Streamlit - Pro e Contro

2024-05-22

Corso di Ingegneria del Software A.A. 2023/2024

github.com/argo-swe

argo-swe.github.io

argo.unipd@gmail.com

Cos’è Streamlit

- **Framework** sviluppato principalmente in Python (back-end) e TypeScript (front-end)
- **React** come libreria open source per la creazione dell’interfaccia utente
- **JSX** (estensione XML-like della sintassi JavaScript) per scrivere “componenti React”
- **Streamlit** consente di trasformare script Python in web app interattive
- Utilizzato per applicazioni di data science e machine learning:
  - Data visualization
  - NLP (Natural Language Processing)
  - LLM e ChatBOT

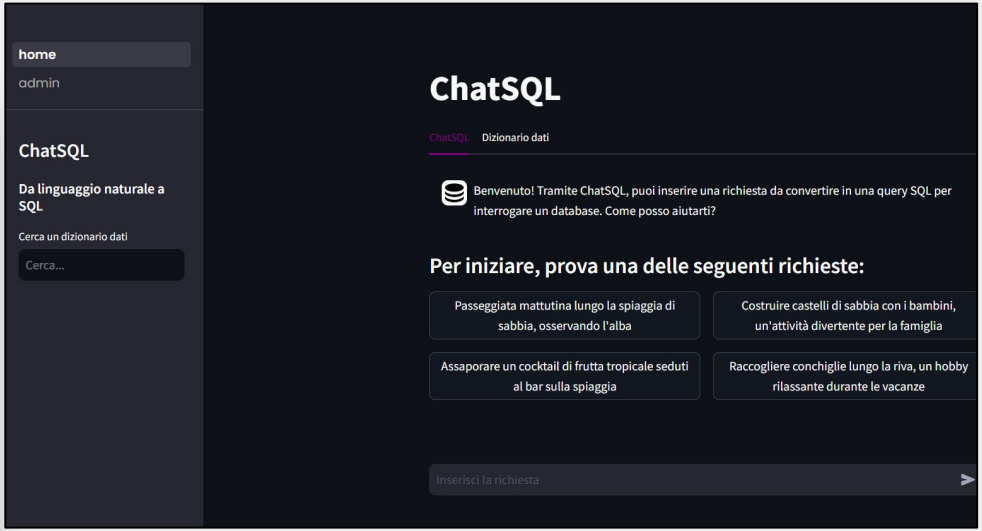
Vantaggi di Streamlit

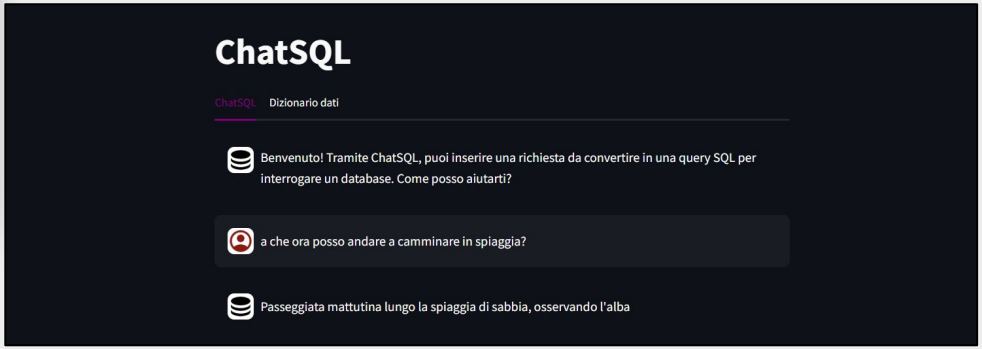
ARGO

- Interattività: built-in components (slider, pulsanti, checkbox, selectbox, grafici) facilmente integrabili
- Aggiungere un widget grafico equivale a dichiarare una variabile
- **Sidebar** = st.sidebar
- **File Uploader** = st.file_uploader
- **Input Chat-GPT like** = st.chat_input(placeholder=”Text”)
- **Connessione a un database SQL** = st.connection("sql")
- Open source
- Comunità di utenti e sviluppatori molto attiva

Vantaggi di Streamlit

- **Rapida prototipazione**: focus sull'analisi dei dati e sull'elaborazione del linguaggio naturale
- **Live editing**: aggiornamento in tempo reale dell'applicazione
- Integrazione con il flusso di lavoro **Python** già esistente
- Impiego di **librerie** (JSON, Pandas, NumPy, timeit) direttamente nell'applicazione
- File di configurazione (config.toml) per modificare alcune proprietà (es.: palette di colori)
- Esecuzione: streamlit run home.py
- Applicazioni facilmente distribuibili





Altre feature di Streamlit

- Facilità nella creazione di app **multipagina**
- Una volta inserite nella cartella pages/, le pagine vengono visualizzate automaticamente nella sidebar
- Il front-end non viene ricaricato ad ogni switch page, rendendo la navigazione più **veloce**
- Di default, Streamlit esegue lo script ogni volta che un utente interagisce con l'app
- Form, dialog, fragment e primitive per il **caching** consentono di modificare tale comportamento e migliorare le performance
- **st.session_state**: condividere le variabili tra le esecuzioni (rerun)

Limitazioni

- Limitata personalizzazione estetica, nonostante la possibilità di incorporare script HTML, CSS e JavaScript
- Framework relativamente nuovo rispetto alla concorrenza: Flask, Django, Vue.js, Next.js e Angular

Alcuni dati

- Nel 2021 è stata rilasciata la versione 1.0 di Streamlit
- Ad oggi, Streamlit conta più di 30.000 stars su GitHub
- Oltre 380k repository dipendono da Streamlit
- La **roadmap** e le feature in sviluppo sono disponibili pubblicamente
- https://roadmap.streamlit.app/

Considerazioni

- Streamlit è nato come un framework il cui scopo principale era scrivere web app semplici in maniera **intuitiva** e **immediata**
- Da qualche anno, invece, le richieste della community hanno portato a una ridefinizione delle ambizioni
- Streamlit è in continuo aggiornamento per rendere le applicazioni più **potenti**, veloci e personalizzabili

Alternative Back-end

- **Flask**
  - Pro: micro-framework leggero e flessibile
  - Pro: curva di apprendimento non particolarmente ripida
  - Contro: mancanza di alcune funzionalità integrate (es.: autenticazione)
- **Django**
  - Pro: vasta gamma di feature “out-of-the-box”
  - Pro: scritto seguendo il paradigma "Model-Template-View" per mantenere una separazione delle responsabilità
  - Pro: scalabilità
  - Contro: flessibilità limitata

Alternative Front-end

ARGO

- **React**
  - Pro: definizione di componenti riutilizzabili
  - Pro: ampia disponibilità di risorse e librerie
  - Pro: JSX, che combina codice di markup e JavaScript
  - Contro: curva di apprendimento ripida
- **Vue.js**
  - Pro: facilità d’uso (template basati su HTML)
  - Pro: reattività integrata
  - Pro: ottime performance
  - Contro: comunità più piccola rispetto a quella di React